



Funciones en SQL

RDBMS: PostgreSQL

Estudiante ID: 24 | Tema: Funciones [24 MOD 4 = 0]

Fecha de entrega: 20 de abril de 2026



Indice

1. Introduccion
2. Que son las funciones en SQL?
3. Tipos de funciones en PostgreSQL
4. Para que sirven las funciones?
5. Contextos de uso
6. Principios y buenas practicas
7. Ejemplo complejo: Sistema de calificaciones estudiantiles
8. Conclusiones
9. Bibliografia



Introduccion

Las bases de datos relacionales van mucho mas alla del simple almacenamiento de informacion. Uno de los recursos mas poderosos que ofrecen los RDBMS son las funciones almacenadas (stored functions), herramientas que permiten encapsular logica directamente en el servidor de base de datos, reutilizar codigo, garantizar consistencia en los calculos y separar la logica de negocio de la capa de aplicacion.

PostgreSQL tiene soporte especialmente robusto para funciones, admitiendo multiples lenguajes de implementacion (PL/pgSQL, SQL puro, PL/Python, PL/Perl) y características avanzadas como funciones de tabla (Set Returning Functions - SRF), funciones de ventana (Window Functions) y funciones agregadas personalizadas. Este documento explora en profundidad que son, como se clasifican, en que contextos se utilizan y cuales son los principios que guian su diseno y uso efectivo.



Que son las funciones en SQL?

Una funcion en SQL es un bloque de codigo almacenado en el servidor de base de datos que:

- Acepta cero o mas parametros de entrada de tipos definidos
- Ejecuta una logica definida (calculos, consultas, validaciones, transformaciones)
- Retorna un valor: escalar, una fila completa, o un conjunto de filas (tabla)
- Puede ser invocada directamente dentro de expresiones SQL (SELECT, WHERE, ORDER BY, JOIN ON, etc.)

A diferencia de los procedimientos almacenados (PROCEDURE), las funciones SIEMPRE retornan un valor y pueden usarse dentro de expresiones SQL como si fueran columnas o valores.

Sintaxis general en PostgreSQL (PL/pgSQL)

```
CREATE OR REPLACE FUNCTION nombre_funcion(param1 TIPO, param2 TIPO)
RETURNS TIPO_RETORNO
LANGUAGE plpgsql
[IMMUTABLE | STABLE | VOLATILE]
AS $$
DECLARE
    variable TIPO;
BEGIN
    -- logica de la funcion
    RETURN valor;
END;
$$;
```



Tipos de funciones en PostgreSQL

3.1 Funciones escalares (Scalar Functions)

Retornan un unico valor primitivo: INTEGER, NUMERIC, TEXT, BOOLEAN, DATE, etc. Son el tipo mas comun. Ejemplo: calcular el promedio ponderado de un alumno devuelve un NUMERIC.

3.2 Funciones de tabla (Set Returning Functions – SRF)

Retornan un conjunto de filas, funcionando como una tabla virtual. Se declaran con RETURNS TABLE(...) o RETURNS SETOF tipo. Pueden usarse en el FROM de una consulta igual que una tabla real.

3.3 Funciones de ventana (Window Functions)

Operan sobre un conjunto de filas relacionadas con la fila actual sin colapsarlas: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), NTILE(), SUM() OVER(...). Son fundamentales para reporteria avanzada.

3.4 Funciones de agregacion integradas

Operan sobre grupos de filas: SUM(), AVG(), COUNT(), MAX(), MIN(), ARRAY_AGG(), STRING_AGG(), JSONB_AGG(). Se usan con GROUP BY.

3.5 Funciones de cadena

Manipulan texto: CONCAT(), SUBSTRING(), UPPER(), LOWER(), LENGTH(), TRIM(), REPLACE(), REGEXP_REPLACE(), FORMAT(), LPAD(), RPAD().

3.6 Funciones de fecha y hora

Operan sobre valores temporales: NOW(), CURRENT_DATE, AGE(), DATE_PART(), EXTRACT(), TO_CHAR(), DATE_TRUNC(), operaciones con INTERVAL.



3.7 Funciones matematicas

Operaciones numericas: `ROUND()`, `FLOOR()`, `CEIL()`, `ABS()`, `MOD()`, `POWER()`, `SQRT()`, `LOG()`, `RANDOM()`, `TRUNC()`.

3.8 Funciones de control de flujo

`CASE WHEN ... THEN ... END`, `COALESCE()`, `NULLIF()`, `GREATEST()`, `LEAST()`. Retornan valores condicionalmente.

3.9 Funciones agregadas personalizadas

PostgreSQL permite crear funciones de agregacion propias con `CREATE AGGREGATE`, definiendo una funcion de transicion (`sfunc`) y opcionalmente una funcion final (`ffunc`). Esto es exclusivo de PostgreSQL y extremadamente poderoso.



Para que sirven las funciones?

- Encapsular logica de negocio repetitiva en el servidor, evitando duplicar codigo en la capa de aplicacion
- Garantizar consistencia: la misma funcion siempre aplica la misma regla de negocio
- Simplificar consultas complejas: una consulta de 50 lineas puede reemplazarse por una llamada a funcion
- Mejorar la seguridad: se otorgan permisos sobre funciones sin exponer las tablas subyacentes (SECURITY DEFINER)
- Mejorar el rendimiento: las funciones IMMUTABLE/STABLE pueden ser cacheadas y optimizadas por el planificador
- Modularidad: facilitan el mantenimiento, los cambios se realizan en un solo lugar
- Abstraccion del dominio: `fn_gpa_estudiante()` es mas expresivo que una subconsulta compleja



Contextos de uso

5.1 Sistemas educativos

Calculo de promedios ponderados, asignacion de calificaciones literales (A/B/C/D/F), determinacion de estado academico (aprobado/reprobado/en recuperacion), calculo de GPA ponderado por creditos, clasificacion de rendimiento estudiantil.

5.2 Sistemas financieros

Calculo de intereses compuestos, amortizaciones de prestamos, conversion de moneda, calculo de impuestos segun tramos (ISR, IVA), comisiones por ventas.

5.3 Recursos humanos

Calculo de salarios netos con deducciones, dias laborados, horas extra, prestaciones legales (aguinaldo, bono 14 en Guatemala), calculo de liquidaciones.

5.4 E-commerce

Calculo de precios con descuento segun categoria de cliente, costos de envio por zona geografica, stock disponible vs reservado, totales de carrito con impuestos.

5.5 Sistemas de salud

Calculo de IMC (indice de masa corporal), dosis ajustadas por peso del paciente, tiempo transcurrido entre citas, alertas de interaccion de medicamentos.

5.6 Reporteria y Business Intelligence

Funciones de ventana para rankings, percentiles, comparacion con periodo anterior con LAG(), calculos de promedios moviles, acumulados progresivos con SUM() OVER().



Principios y buenas practicas

6.1 Volatilidad correcta (IMMUTABLE / STABLE / VOLATILE)

IMMUTABLE: mismos inputs, mismo resultado siempre. No accede a la base de datos. Permite optimizaciones fuertes por parte del planificador. Ejemplo: `fn_nota_a_letra(85.0)` siempre retorna 'B'.

STABLE: mismo resultado dentro de una transaccion. Lee datos pero no los modifica. Ejemplo: `fn_nota_ponderada(id)` lee evaluaciones.

VOLATILE (defecto): puede cambiar en cada llamada. Ejemplo: `NOW()`, `RANDOM()`.

6.2 Responsabilidad unica

Cada funcion resuelve un unico problema. No mezclar calculo de nota con envio de notificaciones.

6.3 Manejo de NULL con COALESCE/NULLIF

Siempre proteger contra valores nulos. `COALESCE(valor, 0)` previene errores silenciosos. `NULLIF(divisor, 0)` previene division por cero.

6.4 Nombres descriptivos con prefijo

Convenio recomendado: `fn_<verbo>_<dominio>()`. Ejemplos: `fn_calcular_gpa_estudiante()`, `fn_obtener_estado_academico()`. Evitar nombres como `fn_calc()` o `fn_procl()`.

6.5 Documentacion con COMMENT ON FUNCTION

Documentar cada funcion directamente en la base de datos:

```
COMMENT ON FUNCTION fn_nota_ponderada(INT)
IS 'Calcula la nota final ponderada de una inscripcion. Retorna
    NUMERIC(5,2).';
```



6.6 Sin efectos secundarios

Las funciones NO deben hacer INSERT/UPDATE/DELETE. Esa responsabilidad corresponde a procedimientos almacenados (PROCEDURE) o triggers.

6.7 Usar esquemas para organizacion

Agrupar funciones por dominio: CREATE FUNCTION
academico.fn_nota_ponderada(...). Facilita permisos y mantenimiento.



Ej. de Sistema de calificaciones estudiantiles

Este ejemplo construye un sistema completo de evaluacion academica en PostgreSQL, con 7 funciones que se llaman entre si para generar reportes completos con funciones de ventana.

7.1 Creacion de tablas

```
-- Tipo ENUM para categorias de evaluacion
CREATE TYPE tipo_evaluacion AS ENUM
    ('parcial1', 'parcial2', 'parcial3', 'proyecto', 'final');

CREATE TABLE estudiantes (
    id_estudiante SERIAL PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    carnet        VARCHAR(20)  UNIQUE NOT NULL,
    fecha_ingreso DATE         NOT NULL
);

CREATE TABLE cursos (
    id_curso      SERIAL PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    creditos      INT           NOT NULL,
    departamento  VARCHAR(50)  NOT NULL
);

CREATE TABLE inscripciones (
    id_inscripcion SERIAL PRIMARY KEY,
    id_estudiante  INT NOT NULL REFERENCES estudiantes,
    id_curso       INT NOT NULL REFERENCES cursos,
    ciclo         VARCHAR(10) NOT NULL
);

CREATE TABLE evaluaciones (
    id_evaluacion SERIAL PRIMARY KEY,
    id_inscripcion INT NOT NULL REFERENCES inscripciones,
    tipo          tipo_evaluacion NOT NULL,
    nota          NUMERIC(5,2) NOT NULL CHECK (nota BETWEEN 0 AND 100),
    peso          NUMERIC(5,2) NOT NULL CHECK (peso BETWEEN 0 AND 100)
```

Juan Carlos Rodríguez López
202231970



```
);
```

7.2 Datos de prueba (INSERT)

Se insertan 3 estudiantes, 3 cursos, 7 inscripciones y 33 evaluaciones con sus respectivos pesos para probar todas las funciones.

```
-- Estudiantes
INSERT INTO estudiantes (nombre, apellido, carnet, fecha_ingreso) VALUES
('Ana', 'Perez', '2021001', '2021-01-15'),
('Luis', 'Garcia', '2021002', '2021-01-15'),
('Maria', 'Lopez', '2021003', '2021-01-15');

-- Cursos (distintos creditos para que el GPA ponderado sea significativo)
INSERT INTO cursos (nombre, creditos, departamento) VALUES
('Bases de Datos I', 4, 'Ingenieria'),
('Programacion Web', 3, 'Ingenieria'),
('Calculo Diferencial', 5, 'Ciencias');

-- Inscripciones: Ana en 3 cursos, Luis en 2, Maria en 2
INSERT INTO inscripciones (id_estudiante, id_curso, ciclo) VALUES
(1,1,'2024-1'), (1,2,'2024-1'), (1,3,'2024-1'),
(2,1,'2024-1'), (2,2,'2024-1'),
(3,1,'2024-1'), (3,3,'2024-1');

-- Evaluaciones (los pesos de cada inscripcion deben sumar 100)
-- Inscripcion 1: Ana en Bases de Datos (5 evaluaciones x 20%)
INSERT INTO evaluaciones (id_inscripcion, tipo, nota, peso) VALUES
(1,'parcial1',78,20), (1,'parcial2',82,20), (1,'parcial3',75,20),
(1,'proyecto',90,20), (1,'final',85,20),

-- Inscripcion 2: Ana en Prog. Web (4 evaluaciones, pesos distintos)
(2,'parcial1',65,25), (2,'parcial2',70,25),
(2,'proyecto',88,25), (2,'final',72,25),

-- Inscripcion 3: Ana en Calculo (esta en riesgo, notas bajas)
(3,'parcial1',55,20), (3,'parcial2',60,20), (3,'parcial3',58,20),
(3,'proyecto',62,20), (3,'final',50,20),

-- Inscripcion 4: Luis en Bases de Datos (excelente rendimiento)
(4,'parcial1',90,20), (4,'parcial2',88,20), (4,'parcial3',92,20),
(4,'proyecto',95,20), (4,'final',93,20),
```



```
-- Inscripcion 5: Luis en Prog. Web (rendimiento satisfactorio)
(5,'parcial1',71,25), (5,'parcial2',68,25),
(5,'proyecto',75,25), (5,'final',80,25),

-- Inscripcion 6: Maria en Bases de Datos (reprobada)
(6,'parcial1',45,20), (6,'parcial2',50,20), (6,'parcial3',48,20),
(6,'proyecto',52,20), (6,'final',40,20),

-- Inscripcion 7: Maria en Calculo (aprobada, compensa el anterior)
(7,'parcial1',88,20), (7,'parcial2',85,20), (7,'parcial3',90,20),
(7,'proyecto',92,20), (7,'final',87,20);
```

Con estos datos, `fn_rendimiento()` retornara resultados variados: Luis sera EXCELENTE, Ana REGULAR (reprobo Calculo), y Maria EN RIESGO ACADEMICO (reprobo Bases de Datos). Esto permite demostrar todos los casos de la logica.

7.3 Funcion 1: `fn_nota_ponderada(id_inscripcion)`

Calcula la nota final ponderada de un estudiante en una inscripcion. Es la funcion base que todas las demas usan.

```
CREATE OR REPLACE FUNCTION fn_nota_ponderada(p_id_inscripcion INT)
RETURNS NUMERIC(5,2)
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_nota NUMERIC(5,2);
BEGIN
    SELECT COALESCE(ROUND(SUM((nota * peso) / 100.0), 2), 0)
    INTO v_nota
    FROM evaluaciones
    WHERE id_inscripcion = p_id_inscripcion;
    RETURN v_nota;
END; $$;
```



7.4 Funcion 2: fn_nota_a_letra(nota)

Convierte una nota numerica (0-100) a su equivalente en letra. Es IMMUTABLE porque no accede a la base de datos.

```
CREATE OR REPLACE FUNCTION fn_nota_a_letra(p_nota NUMERIC(5,2))
RETURNS CHAR(2)
LANGUAGE plpgsql IMMUTABLE
AS $$
BEGIN
    RETURN CASE
        WHEN p_nota >= 90 THEN 'A'
        WHEN p_nota >= 80 THEN 'B'
        WHEN p_nota >= 70 THEN 'C'
        WHEN p_nota >= 60 THEN 'D'
        ELSE 'F'
    END;
END; $$;
```

7.5 Funcion 3: fn_estado_academico(nota)

Determina el estado academico segun la nota final: APROBADO, EN RECUPERACION o REPROBADO.

```
CREATE OR REPLACE FUNCTION fn_estado_academico(p_nota NUMERIC(5,2))
RETURNS VARCHAR(20)
LANGUAGE plpgsql IMMUTABLE
AS $$
BEGIN
    RETURN CASE
        WHEN p_nota >= 70 THEN 'APROBADO'
        WHEN p_nota >= 60 THEN 'EN RECUPERACION'
        ELSE 'REPROBADO'
    END;
END; $$;
```



7.6 Funcion 4: fn_gpa_estudiante(id_estudiante, ciclo)

Calcula el GPA ponderado por creditos del curso. Llama a fn_nota_ponderada internamente. Usa NULLIF para evitar division por cero si no hay inscripciones.

```
CREATE OR REPLACE FUNCTION fn_gpa_estudiante(  
    p_id_estudiante INT, p_ciclo VARCHAR(10))  
RETURNS NUMERIC(5,2) LANGUAGE plpgsql STABLE  
AS $$  
DECLARE v_gpa NUMERIC(5,2);  
BEGIN  
    SELECT COALESCE(ROUND(  
        SUM(fn_nota_ponderada(i.id_inscripcion) * c.creditos)  
        / NULLIF(SUM(c.creditos), 0), 2), 0)  
    INTO v_gpa  
    FROM inscripciones i  
    JOIN cursos c ON i.id_curso = c.id_curso  
    WHERE i.id_estudiante = p_id_estudiante  
        AND i.ciclo = p_ciclo;  
    RETURN v_gpa;  
END; $$;
```

7.7 Funcion 5: fn_cursos_reprobados(id_estudiante, ciclo)

Cuenta cuantos cursos reprobó el estudiante en un ciclo (nota < 70). Llama a fn_nota_ponderada.

```
CREATE OR REPLACE FUNCTION fn_cursos_reprobados(  
    p_id_estudiante INT, p_ciclo VARCHAR(10))  
RETURNS INT LANGUAGE plpgsql STABLE  
AS $$  
DECLARE v_count INT;  
BEGIN  
    SELECT COUNT(*) INTO v_count  
    FROM inscripciones i  
    WHERE i.id_estudiante = p_id_estudiante  
        AND i.ciclo = p_ciclo  
        AND fn_nota_ponderada(i.id_inscripcion) < 70;  
    RETURN v_count;  
END; $$;
```



7.8 Funcion 6: fn_rendimiento(id_estudiante, ciclo)

Clasifica el rendimiento general. Llama a fn_gpa_estudiante y fn_cursos_reprobados – ejemplo de composicion de funciones.

```
CREATE OR REPLACE FUNCTION fn_rendimiento(
    p_id_estudiante INT, p_ciclo VARCHAR(10))
RETURNS VARCHAR(30) LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_gpa          NUMERIC(5,2);
    v_reprobados INT;
BEGIN
    v_gpa          := fn_gpa_estudiante(p_id_estudiante, p_ciclo);
    v_reprobados := fn_cursos_reprobados(p_id_estudiante, p_ciclo);
    RETURN CASE
        WHEN v_reprobados = 0 AND v_gpa >= 85 THEN 'EXCELENTE'
        WHEN v_reprobados = 0 AND v_gpa >= 70 THEN 'SATISFACTORIO'
        WHEN v_reprobados <= 1                     THEN 'REGULAR'
        ELSE 'EN RIESGO ACADEMICO'
    END;
END; $$;
```

7.9 Funcion 7 : fn_reporte_inscripcion – funcion de tabla (SRF)

Retorna un conjunto de filas con el reporte detallado de un estudiante. Incluye funciones de ventana (RANK) para posicion dentro del ciclo.

```
CREATE OR REPLACE FUNCTION fn_reporte_inscripcion(
    p_id_estudiante INT, p_ciclo VARCHAR(10))
RETURNS TABLE (
    curso          VARCHAR,
    creditos       INT,
    nota_final     NUMERIC(5,2),
    letra          CHAR(2),
    estado         VARCHAR(20),
    ranking_ciclo BIGINT
) LANGUAGE plpgsql STABLE AS $$
BEGIN
    RETURN QUERY
    SELECT c.nombre::VARCHAR, c.creditos,
```




```
fn_nota_ponderada(i.id_inscripcion),
fn_nota_a_letra(fn_nota_ponderada(i.id_inscripcion)),
fn_estado_academico(fn_nota_ponderada(i.id_inscripcion)),
RANK() OVER (
    PARTITION BY i.ciclo
    ORDER BY fn_nota_ponderada(i.id_inscripcion) DESC
)
FROM inscripciones i
JOIN cursos c ON i.id_curso = c.id_curso
WHERE i.id_estudiante = p_id_estudiante
    AND i.ciclo = p_ciclo;
END; $$;
```

7.10 Consulta 1: Boleta completa de calificaciones

Esta consulta usa todas las funciones escalares juntas para generar la boleta de un ciclo completo.

```
SELECT
    CONCAT(e.nombre, ' ', e.apellido)      AS estudiante,
    e.carnet,
    c.nombre                                AS curso,
    c.creditos,
    fn_nota_ponderada(i.id_inscripcion)    AS nota_final,
    fn_nota_a_letra(
        fn_nota_ponderada(i.id_inscripcion)) AS letra,
    fn_estado_academico(
        fn_nota_ponderada(i.id_inscripcion)) AS estado,
    fn_gpa_estudiante(e.id_estudiante,'2024-1') AS gpa,
    fn_rendimiento(e.id_estudiante,'2024-1') AS rendimiento
FROM inscripciones i
JOIN estudiantes e ON i.id_estudiante = e.id_estudiante
JOIN cursos c      ON i.id_curso = c.id_curso
WHERE i.ciclo = '2024-1'
ORDER BY e.apellido, e.nombre, c.nombre;
```



7.11 Consulta 2: Reporte con funciones de ventana (ranking y percentil)

Consulta avanzada que incluye RANK() y PERCENT_RANK() para mostrar el puesto y percentil de cada estudiante.

```
SELECT
  CONCAT(e.nombre, ' ', e.apellido) AS estudiante,
  fn_gpa_estudiante(e.id_estudiante, '2024-1') AS gpa,
  fn_nota_a_letra(
    fn_gpa_estudiante(e.id_estudiante, '2024-1')) AS letra_gpa,
  fn_cursos_reprobados(e.id_estudiante, '2024-1') AS reprobados,
  fn_rendimiento(e.id_estudiante, '2024-1') AS clasificacion,
  RANK() OVER (
    ORDER BY fn_gpa_estudiante(e.id_estudiante, '2024-1') DESC
  ) AS puesto,
  ROUND(
    (PERCENT_RANK() OVER (
      ORDER BY fn_gpa_estudiante(e.id_estudiante, '2024-1')
    ) * 100)::NUMERIC, 1
  ) AS percentil
FROM (
  SELECT DISTINCT id_estudiante FROM inscripciones
  WHERE ciclo = '2024-1'
) base
JOIN estudiantes e ON base.id_estudiante = e.id_estudiante
ORDER BY gpa DESC;
```

7.12 Usando la funcion de tabla (SRF)

Las funciones de tabla se usan en el FROM como si fueran tablas reales, pudiendo unirse con otras tablas.

```
-- Reporte del estudiante con ID 1
SELECT * FROM fn_reporte_inscripcion(1, '2024-1');

-- Union con datos del estudiante
SELECT e.nombre, e.carnet, r.*
FROM fn_reporte_inscripcion(1, '2024-1') r
JOIN estudiantes e ON e.id_estudiante = 1;
```



Imágenes del ejemplo

8.1 Creación de tablas

```
Query Query History
1  -- Tipo ENUM para categorías de evaluación
2  CREATE TYPE tipo_evaluacion AS ENUM
3  ('parcial1','parcial2','parcial3','proyecto','final');
4
5  CREATE TABLE estudiantes (
6  id_estudiante SERIAL PRIMARY KEY,
7  nombre VARCHAR(100) NOT NULL,
8  apellido VARCHAR(100) NOT NULL,
9  carnet VARCHAR(20) UNIQUE NOT NULL,
10 fecha_ingreso DATE NOT NULL
11 );
12
13 CREATE TABLE cursos (
14 id_curso SERIAL PRIMARY KEY,
15 nombre VARCHAR(100) NOT NULL,
16 creditos INT NOT NULL,
17 departamento VARCHAR(50) NOT NULL
18 );
19
20 CREATE TABLE inscripciones (
21 id_inscripcion SERIAL PRIMARY KEY,
22 id_estudiante INT NOT NULL REFERENCES estudiantes,
23 id_curso INT NOT NULL REFERENCES cursos,
24 ciclo VARCHAR(10) NOT NULL
25 );
26
27 CREATE TABLE evaluaciones (
28 id_evaluacion SERIAL PRIMARY KEY,
29 id_inscripcion INT NOT NULL REFERENCES inscripciones,
30 tipo tipo_evaluacion NOT NULL,
31 nota NUMERIC(5,2) NOT NULL CHECK (nota BETWEEN 0 AND 100),
32 peso NUMERIC(5,2) NOT NULL CHECK (peso BETWEEN 0 AND 100)
33 );

Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 74 msec.
Total rows: Query complete 00:00:00.074
```



8.2 Datos de prueba (INSERT)

```
Query Query History
1
2 -- Estudiantes
3 INSERT INTO estudiantes (nombre, apellido, carnet, fecha_ingreso) VALUES
4 ('Ana', 'Perez', '2021001', '2021-01-15'),
5 ('Luis', 'Garcia', '2021002', '2021-01-15'),
6 ('Maria', 'Lopez', '2021003', '2021-01-15');
7
8 -- Cursos (distintos creditos para que el GPA ponderado sea significativo)
9 INSERT INTO cursos (nombre, creditos, departamento) VALUES
10 ('Bases de Datos I', 4, 'Ingenieria'),
11 ('Programacion Web', 3, 'Ingenieria'),
12 ('Calculo Diferencial', 5, 'Ciencias');
13
14 -- Inscripciones: Ana en 3 cursos, Luis en 2, Maria en 2
15 INSERT INTO inscripciones (id_estudiante, id_curso, ciclo) VALUES
16 (1,1,'2024-1'), (1,2,'2024-1'), (1,3,'2024-1'),
17 (2,1,'2024-1'), (2,2,'2024-1'),
18 (3,1,'2024-1'), (3,3,'2024-1');
19
20 -- Evaluaciones (los pesos de cada inscripcion deben sumar 100)
21 -- Inscripcion 1: Ana en Bases de Datos (5 evaluaciones x 20%)
22 INSERT INTO evaluaciones (id_inscripcion, tipo, nota, peso) VALUES
23 (1,'parcial1',78,20),(1,'parcial2',82,20),(1,'parcial3',75,20),
24 (1,'proyecto',90,20),(1,'final',85,20),
25
26 -- Inscripcion 2: Ana en Prog. Web (4 evaluaciones, pesos distintos)
27 (2,'parcial1',65,25),(2,'parcial2',70,25),
28 (2,'proyecto',88,25),(2,'final',72,25),
29
30 -- Inscripcion 3: Ana en Calculo (esta en riesgo, notas bajas)
31 (3,'parcial1',55,20),(3,'parcial2',60,20),(3,'parcial3',50,20),
32 (3,'proyecto',62,20),(3,'final',50,20),
33
Data Output Messages Notifications
INSERT 0 33
Query returned successfully in 70 msec.
Total rows: Query complete 00:00:00.070
```

datos estudiantes

	id_estudiante [PK] integer	nombre character varying (100)	apellido character varying (100)	carnet character varying (20)	fecha_ingreso date
1	1	Ana	Perez	2021001	2021-01-15
2	2	Luis	Garcia	2021002	2021-01-15
3	3	Maria	Lopez	2021003	2021-01-15

datos cursos

	id_curso [PK] integer	nombre character varying (100)	creditos integer	departamento character varying (50)
1	1	Bases de Datos I	4	Ingenieria
2	2	Programacion Web	3	Ingenieria
3	3	Calculo Diferencial	5	Ciencias

Juan Carlos Rodríguez López
202231970



datos evaluaciones:

	Id_evaluacion [PK] integer	Id_Inscripcion integer	tipo tipo_evaluacion	nota numeric (5,2)	peso numeric (5,2)
1	1	1	parcial1	78.00	20.00
2	2	1	parcial2	82.00	20.00
3	3	1	parcial3	75.00	20.00
4	4	1	proyecto	90.00	20.00
5	5	1	final	85.00	20.00
6	6	2	parcial1	65.00	25.00
7	7	2	parcial2	70.00	25.00
8	8	2	proyecto	88.00	25.00
9	9	2	final	72.00	25.00
10	10	3	parcial1	55.00	20.00
11	11	3	parcial2	60.00	20.00
12	12	3	parcial3	58.00	20.00
13	13	3	proyecto	62.00	20.00
14	14	3	final	50.00	20.00
15	15	4	parcial1	90.00	20.00
16	16	4	parcial2	88.00	20.00
17	17	4	parcial3	92.00	20.00
18	18	4	proyecto	95.00	20.00
19	19	4	final	93.00	20.00
20	20	5	parcial1	71.00	25.00
21	21	5	parcial2	68.00	25.00
22	22	5	proyecto	75.00	25.00
23	23	5	final	80.00	25.00
24	24	6	parcial1	45.00	20.00
25	25	6	parcial2	50.00	20.00
26	26	6	parcial3	48.00	20.00

datos inscripción:

	Id_Inscripcion [PK] integer	Id_estudiante integer	Id_curso integer	ciclo character varying (10)
1	1	1	1	2024-1
2	2	1	2	2024-1
3	3	1	3	2024-1
4	4	2	1	2024-1
5	5	2	2	2024-1
6	6	3	1	2024-1
7	7	3	3	2024-1



8.3 Funcion 1: fn_nota_ponderada(id_inscripcion)

```
Query  Query History
1  CREATE OR REPLACE FUNCTION fn_nota_ponderada(p_id_inscripcion INT)
2  RETURNS NUMERIC(5,2)
3  LANGUAGE plpgsql STABLE
4  AS $$
5  DECLARE
6      v_nota NUMERIC(5,2);
7  BEGIN
8      SELECT COALESCE(ROUND(SUM((nota * peso) / 100.0), 2), 0)
9      INTO v_nota
10     FROM evaluaciones
11     WHERE id_inscripcion = p_id_inscripcion;
12     RETURN v_nota;
13 END; $$
14
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 47 msec.

8.4 Funcion 2: fn_nota_a_letra(nota)

```
Query  Query History
1  CREATE OR REPLACE FUNCTION fn_nota_a_letra(p_nota NUMERIC(5,2))
2  RETURNS CHAR(2)
3  LANGUAGE plpgsql IMMUTABLE
4  AS $$
5  BEGIN
6      RETURN CASE
7          WHEN p_nota >= 90 THEN 'A'
8          WHEN p_nota >= 80 THEN 'B'
9          WHEN p_nota >= 70 THEN 'C'
10         WHEN p_nota >= 60 THEN 'D'
11         ELSE 'F'
12     END;
13 END; $$
14
15
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 59 msec.



8.5 Funcion 3: fn_estado_academico(nota)

```
Query  Query History
1  CREATE OR REPLACE FUNCTION fn_estado_academico(p_nota NUMERIC(5,2))
2  RETURNS VARCHAR(20)
3  LANGUAGE plpgsql IMMUTABLE
4  AS $$
5  BEGIN
6      RETURN CASE
7          WHEN p_nota >= 70 THEN 'APROBADO'
8          WHEN p_nota >= 60 THEN 'EN RECUPERACION'
9          ELSE 'REPROBADO'
10     END;
11 END; $$;
12
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 58 msec.

8.6 Funcion 4: fn_gpa_estudiante(id_estudiante, ciclo)

```
Query  Query History
1  CREATE OR REPLACE FUNCTION fn_gpa_estudiante(
2  p_id_estudiante INT, p_ciclo VARCHAR(10))
3  RETURNS NUMERIC(5,2) LANGUAGE plpgsql STABLE
4  AS $$
5  DECLARE v_gpa NUMERIC(5,2);
6  BEGIN
7      SELECT COALESCE(ROUND(
8          SUM(fn_nota_ponderada(i.id_inscripcion) * c.creditos)
9          / NULLIF(SUM(c.creditos), 0), 2), 0)
10     INTO v_gpa
11     FROM inscripciones i
12     JOIN cursos c ON i.id_curso = c.id_curso
13     WHERE i.id_estudiante = p_id_estudiante
14           AND i.ciclo = p_ciclo;
15     RETURN v_gpa;
16 END; $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 54 msec.



8.7 Funcion 5: fn_cursos_reprobados(id_estudiante, ciclo)

```
Query Query History
1 CREATE OR REPLACE FUNCTION fn_cursos_reprobados(
2   p_id_estudiante INT, p_ciclo VARCHAR(10))
3 RETURNS INT LANGUAGE plpgsql STABLE
4 AS $$
5 DECLARE v_count INT;
6 BEGIN
7   SELECT COUNT(*) INTO v_count
8   FROM inscripciones i
9   WHERE i.id_estudiante = p_id_estudiante
10        AND i.ciclo = p_ciclo
11        AND fn_nota_ponderada(i.id_inscripcion) < 70;
12   RETURN v_count;
13 END; $$
14
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 43 msec.

8.8 Funcion 6: fn_rendimiento(id_estudiante, ciclo)

```
Query Query History
1 CREATE OR REPLACE FUNCTION fn_rendimiento(
2   p_id_estudiante INT, p_ciclo VARCHAR(10))
3 RETURNS VARCHAR(30) LANGUAGE plpgsql STABLE
4 AS $$
5 DECLARE
6   v_gpa NUMERIC(5,2);
7   v_reprobados INT;
8 BEGIN
9   v_gpa := fn_gpa_estudiante(p_id_estudiante, p_ciclo);
10  v_reprobados := fn_cursos_reprobados(p_id_estudiante, p_ciclo);
11  RETURN CASE
12    WHEN v_reprobados = 0 AND v_gpa >= 85 THEN 'EXCELENTE'
13    WHEN v_reprobados = 0 AND v_gpa >= 70 THEN 'SATISFACTORIO'
14    WHEN v_reprobados <= 1 THEN 'REGULAR'
15    ELSE 'EN RIESGO ACADEMICO'
16  END;
17 END; $$
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 41 msec.



8.9 Funcion 7: fn_reporte_inscripcion – funcion de tabla (SRF)

```
Query Query History
1 CREATE OR REPLACE FUNCTION fn_reporte_inscripcion(
2   p_id_estudiante INT, p_ciclo VARCHAR(10))
3 RETURNS TABLE (
4   curso          VARCHAR,
5   creditos       INT,
6   nota_final     NUMERIC(5,2),
7   letra          CHAR(2),
8   estado         VARCHAR(20),
9   ranking_ciclo  BIGINT
10 ) LANGUAGE plpgsql STABLE AS $$
11 BEGIN
12 RETURN QUERY
13 SELECT c.nombre::VARCHAR, c.creditos,
14        fn_nota_ponderada(i.id_inscripcion),
15        fn_nota_a_letra(fn_nota_ponderada(i.id_inscripcion)),
16        fn_estado_academico(fn_nota_ponderada(i.id_inscripcion)),
17        RANK() OVER (
18          PARTITION BY i.ciclo
19          ORDER BY fn_nota_ponderada(i.id_inscripcion) DESC
20        )
21 FROM inscripciones i
22 JOIN cursos c ON i.id_curso = c.id_curso
23 WHERE i.id_estudiante = p_id_estudiante
24        AND i.ciclo = p_ciclo;
25 END; $$;
Data Output Messages Notifications
CREATE FUNCTION
Query returned successfully in 52 msec.
```

8.10 Consulta 1: Boleta completa de calificaciones

Query Query History

```

1  SELECT
2  CONCAT(e.nombre, ' ', e.apellido) AS estudiante,
3  e.carnet,
4  c.nombre AS curso,
5  c.creditos,
6  fn_nota_ponderada(i.id_inscripcion) AS nota_final,
7  fn_nota_a_letra(
8  fn_nota_ponderada(i.id_inscripcion)) AS letra,
9  fn_estado_academico(
10 fn_nota_ponderada(i.id_inscripcion)) AS estado,
11 fn_gpa_estudiante(e.id_estudiante, '2024-1') AS gpa,
12 fn_rendimiento(e.id_estudiante, '2024-1') AS rendimiento
13 FROM inscripciones i
14 JOIN estudiantes e ON i.id_estudiante = e.id_estudiante
15 JOIN cursos c ON i.id_curso = c.id_curso
16 WHERE i.ciclo = '2024-1'
17 ORDER BY e.apellido, e.nombre, c.nombre;

```

Data Output Messages Notifications

	estudiante	carnet	curso	creditos	nota_final	letra	estado	gpa	rendimiento
	estudiante	character varying (20)	curso	integer	numeric	character	estado	character varying	character varying
1	Luis Garcia	2021002	Bases de Datos I	4	91.60	A	APROBADO	83.84	SATISFACTORIO
2	Luis Garcia	2021002	Programacion Web	3	73.50	C	APROBADO	83.84	SATISFACTORIO
3	Maria Lopez	2021003	Bases de Datos I	4	47.00	F	REPROBADO	70.00	REGULAR
4	Maria Lopez	2021003	Calculo Diferencial	5	88.40	B	APROBADO	70.00	REGULAR
5	Ana Perez	2021001	Bases de Datos I	4	82.00	B	APROBADO	69.52	REGULAR
6	Ana Perez	2021001	Calculo Diferencial	5	57.00	F	REPROBADO	69.52	REGULAR
7	Ana Perez	2021001	Programacion Web	3	73.75	C	APROBADO	69.52	REGULAR

8.11 Consulta 2: Reporte con funciones de ventana (ranking y percentil)

Query
Query History

```

1  SELECT
2      CONCAT(e.nombre, ' ', e.apellido) AS estudiante,
3      fn_gpa_estudiante(e.id_estudiante,'2024-1') AS gpa,
4      fn_notas_a_letra(
5          fn_gpa_estudiante(e.id_estudiante,'2024-1')) AS letra_gpa,
6      fn_cursos_reprobados(e.id_estudiante,'2024-1') AS reprobados,
7      fn_rendimiento(e.id_estudiante,'2024-1') AS clasificacion,
8      RANK() OVER (
9          ORDER BY fn_gpa_estudiante(e.id_estudiante,'2024-1') DESC
10     ) AS puesto,
11     ROUND(
12         (PERCENT_RANK() OVER (
13             ORDER BY fn_gpa_estudiante(e.id_estudiante,'2024-1')
14         ) * 100)::NUMERIC, 1
15     ) AS percentil
16 FROM (
17     SELECT DISTINCT id_estudiante FROM inscripciones
18     WHERE ciclo = '2024-1'
19 ) base
20 JOIN estudiantes e ON base.id_estudiante = e.id_estudiante
21 ORDER BY gpa DESC;

```

Data Output
Messages
Notifications

≡+
📄
▼
🗑️
▼
🗑️
📁
📄
SQL

	estudiante text	gpa numeric	letra_gpa character	reprobados integer	clasificacion character varying	puesto bigint	percentil numeric
1	Luis Garcia	83.84	B	0	SATISFACTORIO	1	100.0
2	Maria Lopez	70.00	C	1	REGULAR	2	50.0
3	Ana Perez	69.52	D	1	REGULAR	3	0.0

8.12 Usando la funcion de tabla (SRF)

```
Query    Query History

1  -- Reporte del estudiante con ID 1
2  SELECT * FROM fn_reporte_inscripcion(1, '2024-1');
3
4  -- Union con datos del estudiante
5  SELECT e.nombre, e.carnet, r.*
6  FROM fn_reporte_inscripcion(1, '2024-1') r
7  JOIN estudiantes e ON e.id_estudiante = 1;
8

Data Output  Messages  Notifications

[Icons] [SQL]

nombre character varying (100)  carnet character varying (20)  curso character varying  creditos integer  nota_final numeric  letra character  estado character varying  ranking_ciclo bigint
1 Ana  2021001  Bases de Datos I  4  82.00  B  APROBADO  1
2 Ana  2021001  Programacion Web  3  73.75  C  APROBADO  2
3 Ana  2021001  Calculo Diferencial  5  57.00  F  REPROBADO  3
```



Conclusiones

A lo largo de esta investigación se demostró que las funciones en PostgreSQL representan uno de los pilares fundamentales del desarrollo de bases de datos relacionales maduras. Los principales hallazgos son:

1. Las funciones permiten encapsular y reutilizar lógica de negocio compleja directamente en el servidor, eliminando redundancia entre la capa de aplicación y la base de datos.
2. La capacidad de PostgreSQL para funciones es superior a la de otros RDBMS: admite múltiples lenguajes, funciones de tabla (SRF), funciones de ventana y funciones agregadas personalizadas.
3. El sistema de volatilidad (IMMUTABLE / STABLE / VOLATILE) permite al planificador de consultas optimizar inteligentemente la ejecución, siendo especialmente valioso en consultas de alto volumen.
4. El anidamiento de funciones (fn_rendimiento llama a fn_gpa_estudiante y fn_cursos_reprobados, que llama a fn_nota_ponderada) demostró como construir sistemas modulares y mantenibles con código limpio.
5. Las funciones de tabla (SRF) con RETURNS TABLE(...) amplían enormemente la utilidad, permitiendo retornar conjuntos de resultados que se comportan como tablas virtuales y pueden unirse con otras tablas.
6. Dominar las funciones en PostgreSQL eleva significativamente la calidad, seguridad y mantenibilidad de cualquier sistema de información que use bases de datos relacionales.



Bibliografia

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: PL/pgSQL - SQL Procedural Language.
<https://www.postgresql.org/docs/current/plpgsql.html>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: CREATE FUNCTION.
<https://www.postgresql.org/docs/current/sql-createfunction.html>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Coronel, C., Morris, S., & Rob, P. (2018). Database Systems: Design, Implementation & Management (13th ed.). Cengage Learning.

Obe, R., & Hsu, L. (2017). PostgreSQL: Up and Running (3rd ed.). O'Reilly Media.

Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.